AMERICAN INTERNATIONAL UNIVARSITY BANGLADESH (AIUB)

Faculty of Science and Technology

Department of Computer Science and Engineering

CSC2210 : OBJECT ORIENTED PROGRAMMING 2

Summer 2025-2026

PROJECT REPORT

STUDENT MANAGEMENT SYSTEM

Supervised by

Dr. Md. Iftekharul Mobin

Submitted By

Name	Id
Shahriar Nafiz Nahin	24-59162-3
Badhon Kabiraj	24-58369-2
Nabila Islam	24-57371-2
ARIFA ASHRAFY	24-57193-2

Table Of Contents

CO2 / CO3 ASSESMENT CRITERIA

Marks are intentionally left blank , as required by the submission instructions

CO2 Criteria	0	1-2	3	4	5	Marks
Requirement fulfillment						
Validation						
Verification						

CO2 Criteria	0	1-2	3	4	5	Marks
Organization of the application						
Representation and integration of database						
Graphical User Interface						

1.Introduction and case study

In many coaching centers today, heavy reliance on fragmented manual paperwork and legacy record-keeping — admission forms, attendance/marks registers, and filing cabinets — creates severe administrative bottlenecks, making it arduous to track student registrations, record exam scores, and provide learners with transparent access to their academic progress. Without an integrated digital solution, administrative staff suffer from disorganized data and overwhelming workloads, while students are left in the dark regarding their marks and profile updates.

To solve this, our **Student Management System** for **Mdemy SSC Math Care** introduces a centralized digital ecosystem designed around three distinct actors: **Students**, who want seamless, real-time access to view their profiles, confirm their enrollment status, and monitor their exam scores in Math; **Admins**, who need efficient tools to register new students within the institution's fixed capacity, record marks, and generate printable institutional reports; and the **Super Admin**, who requires top-level authority to oversee system operations, create new admin accounts, and control system-wide permissions.

Because Mdemy SSC Math Care teaches a single subject — **SSC Math** — exclusively to SSC candidates, with a fixed capacity of **150–200 students**, the system does away with a multi-course catalog entirely. At the core of this environment exist three primary entities: **Students**, who are uniquely identified by their username and credentials while holding detailed personal attributes such as first and last name, phone number, date of birth, address, gender, and profile photo; **Scores**, which capture the specific performance marks obtained by a student in SSC Math; and **Admins**, whose profiles are defined by administrative credentials, contact details, and an active or inactive status flag.

These entities interact through well-defined relational pathways: every registered student is automatically associated with the single SSC Math subject — there is no course selection or enrollment step — and individual score records link each student to their Math performance, while the Super Admin maintains hierarchical governance over all Admin accounts. Registration itself is capacity-gated: the system enforces the institution's 150–200 student limit at the point of admission, rejecting new registrations once capacity is reached. Financially, the system accommodates institutional revenue flow by tracking the single tuition/registration fee paid by each student and processed through administrative oversight, ensuring clear financial tracking for the institution without involving external third-party platform commissions.

2 .Functional Requirements & User Stories

(a) Functional Requirement List

Admin & Super Admin

1. Admin can register new students by capturing personal, contact, and profile image data.

2. Admin can view, search, update, and delete student records.
3. Admin can generate and print reports containing student information.
4. Admin can enroll students into specific courses.
5. Admin can create new courses with duration and hourly details.
6. Admin can manage existing courses through search, update, and delete operations.
7. Admin can generate and print course reports.
8. Admin can record individual student scores for specific courses.
9. Admin can manage score records via search, update, and delete operations.
10. Admin can generate and print result reports for individual students or courses.

Super Admin Only 11. Super Admin can create new Admin accounts with secure credentials and status toggles. 12. Super Admin can manage (view, search, update, delete, activate/deactivate) existing Admin accounts. 13. Super Admin can generate and print Admin account reports.

Student 14. Student can view their own profile information. 15. Student can update their personal profile details. 16. Student can view their course scores and academic results. 17. Student can print their profile and score summaries.

(b) User Stories

As an Admin/Super Admin, I can register a new student, so that they can be tracked in the system. Details: The Admin clicks "Student Registration" to open a form containing fields for First Name, Last Name, username, password, phone, DOB, address, gender, and photo upload. The system validates that all fields are filled and the username is unique before saving. On click of the "Save" button, a new record is inserted into the Students table, and the form resets.

As an Admin/Super Admin, I can manage student records, so that data remains accurate and up-to-date. Details: The Admin navigates to "Manage Student" and uses a search bar to filter students by name or ID. A list is displayed in a DataGridView, where the Admin can select a row to either "Update" or "Delete" the information. Once an action is confirmed, the database updates or removes the record, and the UI refreshes to show the current state.

As an Admin/Super Admin, I can enroll a student in a course, so that their academic path is defined. Details: The Admin selects a student and a course from respective dropdown menus under the "Enroll To Course" section. The system checks if the student is already enrolled in that course to prevent duplicates. Upon clicking "Enroll," the relationship is stored in the Enrollment table, linking the StudentID and CourseID.

As an Admin/Super Admin, I can add a new course, so that the institution can offer new subjects. Details: The Admin opens the "Add Course" form which requires Course Name, Course Duration, and Course Hours. The system validates that Duration and Hours are positive integers. Upon submission, a new entry is added to the Courses table, making the course available for student enrollment.

As an Admin/Super Admin, I can record a student's score, so that their performance is documented. Details: The Admin selects the target student and the relevant course from dropdown lists, then enters the numeric marks in the "New Score" section. The system validates that the marks are within a valid

range (0-100). When "Save" is clicked, the score is saved in the Scores table, associating the student, course, and grade.

As a Super Admin, I can add a new Admin, so that administrative duties can be delegated. Details: The Super Admin fills out a form requiring First Name, Last Name, Username, Password, Phone, Email, Address, and an Active/Inactive status. The system validates that the username and email are not already in use. Upon clicking "Create," a new user record is inserted into the Admin/Super Admin table with the assigned status.

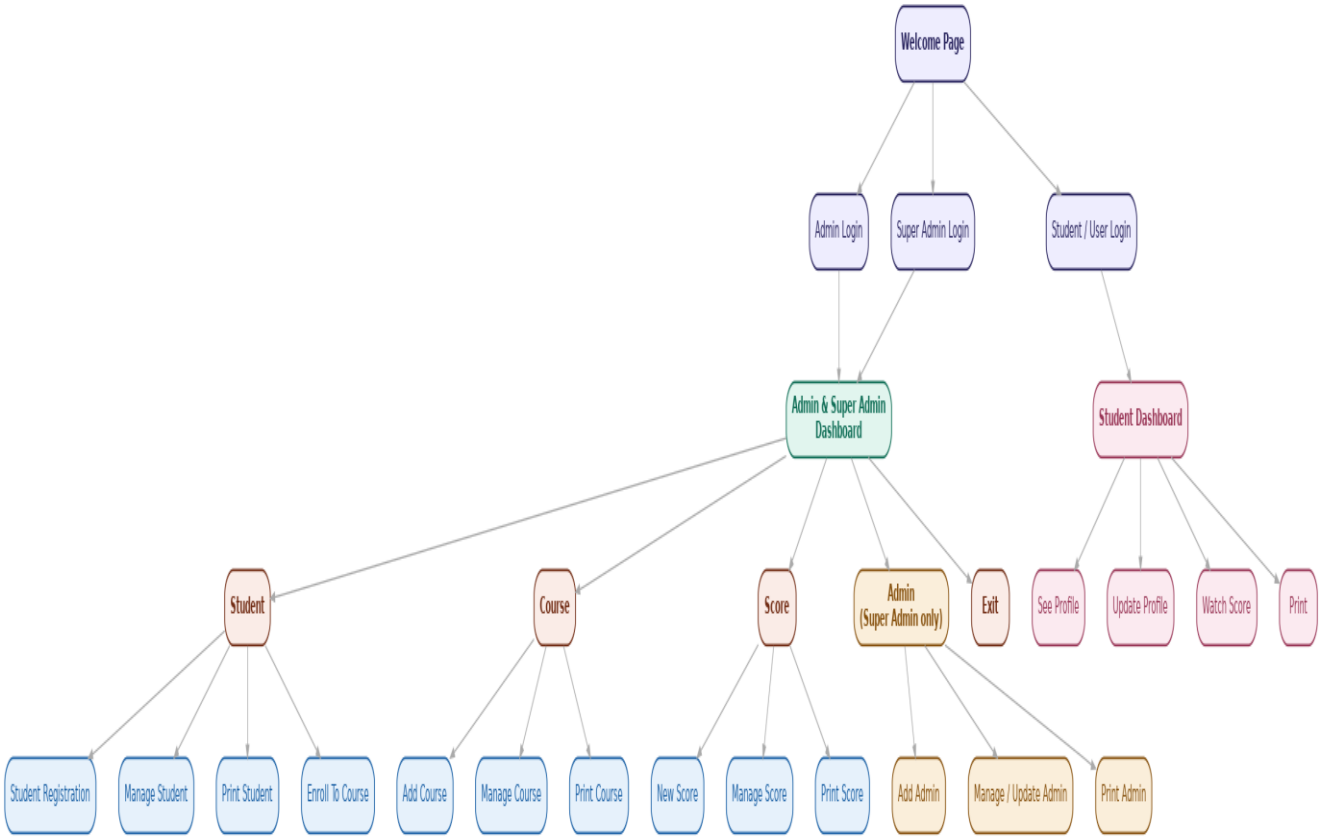
As a Student, I can view my profile, so that I can verify my personal information. Details: Upon logging in, the Student navigates to the "See Profile" section. The system retrieves the student's data from the database based on their logged-in UserID and displays it in a read-only layout. This provides the student with an instant overview of their registered details.

As a Student, I can update my profile, so that my contact information remains current. Details: The Student clicks "Update Profile," which populates editable fields with their existing data. The student can modify fields such as Phone, Address, or Password. Upon clicking "Save," the system validates the new input, updates the database record, and displays a success confirmation.

As a Student, I can watch my scores, so that I can track my academic progress. Details: The Student navigates to "Watch Score" to view a list of all their enrolled courses and associated grades. The system pulls data from the Scores and Enrollment tables based on the StudentID. The results are displayed in a formatted table or list for easy readability.

As a Student, I can print my records, so that I have a physical copy of my results. Details: The Student selects "Print" from the dashboard, which opens a print preview window generated from the system data. This allows the student to send their academic summary or profile details to a local printer. The printout is formatted professionally for clarity.

3 . UI Navigation Diagram



4 . Database Design

*SQL Schema Diagram

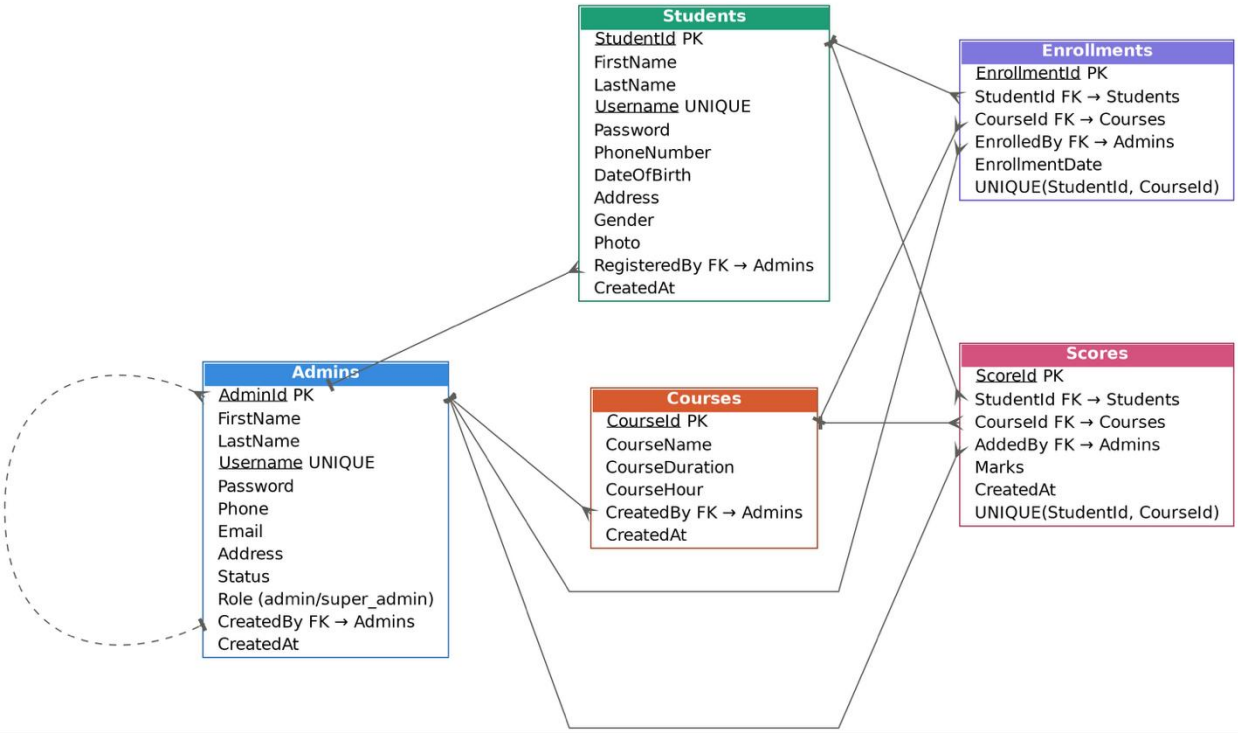
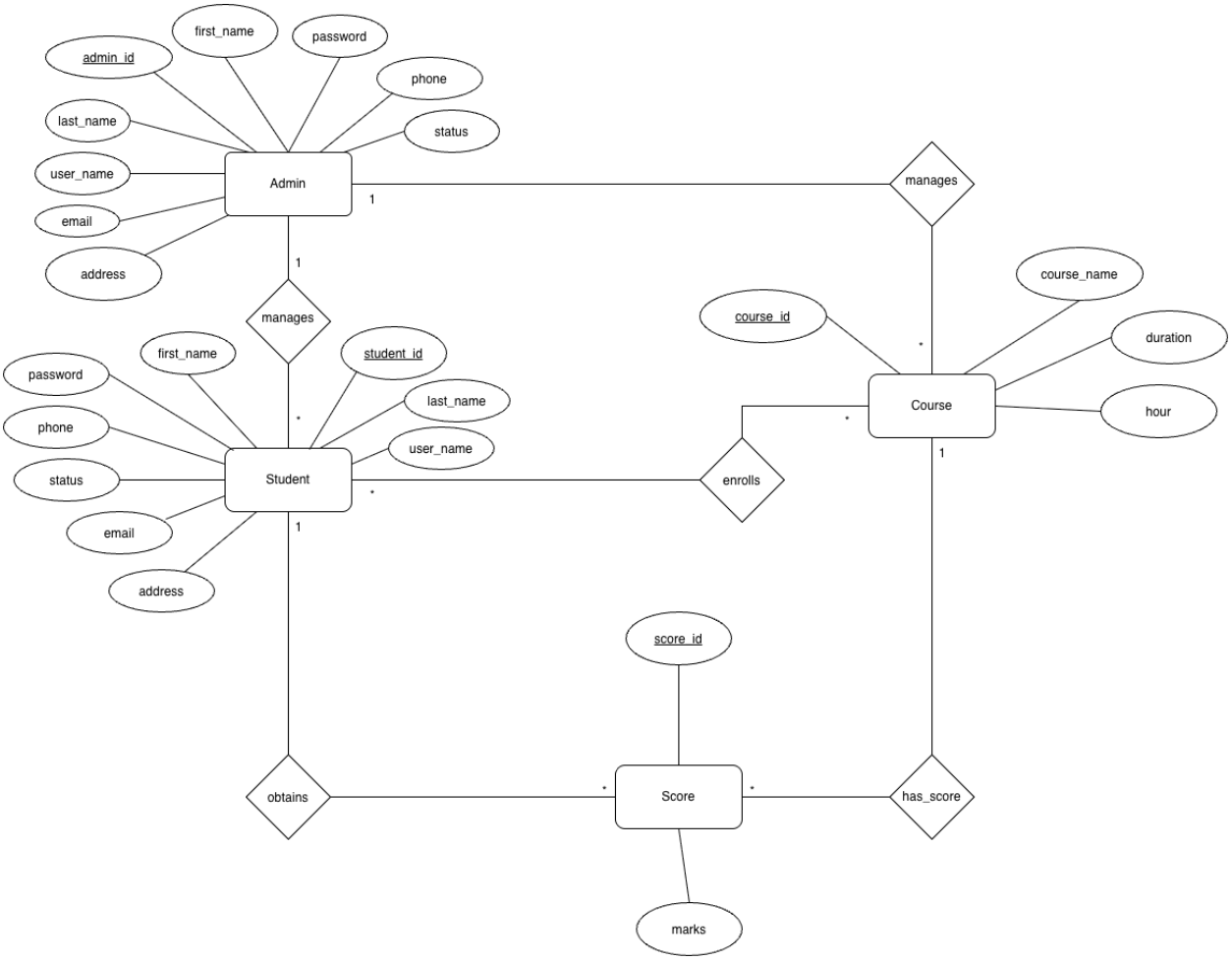


Table	Purpose	Key columns
Admins	Both Admin & Super Admin in one table	AdminId PK, FirstName, LastName, Username UNIQUE, Password, Phone, Email, Address, Status, Role (admin/super_admin), CreatedBy FK→Admins, CreatedAt
Students	One row per registered student	StudentId PK, FirstName, LastName, Username UNIQUE, Password, PhoneNumber, DateOfBirth, Address, Gender, Photo, RegisteredBy FK→Admins, CreatedAt
Courses	Courses offered by the institution	CourseId PK, CourseName, CourseDuration, CourseHour, CreatedBy FK→Admins, CreatedAt
Enrollments	Junction table — student ↔ course	EnrollmentId PK, StudentId FK→Students, CourseId FK→Courses, EnrolledBy FK→Admins, EnrollmentDate, UNIQUE(StudentId, CourseId)
Scores	Marks per student per course	ScoreId PK, StudentId FK→Students, CourseId FK→Courses, Marks, AddedBy FK→Admins, CreatedAt, UNIQUE(StudentId, CourseId)

5.ER Diagram



6.Normalization & Finalization

UNF: (student_id, first_name, last_name, username, password, phone, address, status,email,course_id, course_name, duration, hour, admin_id,first_name,last_name, user_name, password, phone,email, address, status)

1NF:

UNF:student_id, first_name, last_name, username, password, phone, address.

1NF: student_id, course_id, course_name, duration, hours.

2NF: admin_id, first_name, last_name, username, password, phone, email, address, status.

2NF:

UNF:student_id, first_name, last_name, username, password, phone, address.

1NF: course_id, course_name, duration, hours.

2NF: *student_id*, *course_id*,admin_id

3NF: score_id, *student_id*, *course_id*.

3NF:

Same as 2NF

Finalization:

1. student_id, first_name, last_name, username, password, phone, address.
2. student_id, course_id, course_name, duration, hours.
3. admin_id, first_name, last_name, username, password, phone, email, address, status.
4. *student_id*, *course_id*,admin_id
5. course_id, course_name, duration, hours.
6. score_id, *student_id*, *course_id*.

7. SQL Queries

Admins

```
INSERT INTO Admins (FirstName, LastName, Username, Password, Phone, Email, Address, Status, Role, CreatedBy, CreatedAt)
VALUES ('John', 'Doe', 'johndoe', 'hashedpassword', '01700000000', 'john@example.com', 'Dhaka', 'active', 'admin', 1, NOW());

SELECT * FROM Admins;

SELECT * FROM Admins WHERE AdminId = 1;

SELECT * FROM Admins WHERE Role = 'super_admin';

SELECT * FROM Admins WHERE Status = 'active';

UPDATE Admins SET Phone = '01812345678', Status = 'inactive' WHERE AdminId = 1;

DELETE FROM Admins WHERE AdminId = 1;

SELECT Role, COUNT(*) AS TotalAdmins FROM Admins GROUP BY Role;
```

Students

```
INSERT INTO Students (FirstName, LastName, Username, Password, PhoneNumber, DateOfBirth, Address, Gender, Photo, RegisteredBy, CreatedAt)
VALUES ('Alice', 'Khan', 'alicek', 'hashedpassword', '01911111111', '2005-05-10', 'Dhaka', 'Female', 'alice.jpg', 1, NOW());

SELECT * FROM Students;

SELECT * FROM Students WHERE StudentId = 1;

SELECT * FROM Students WHERE RegisteredBy = 1;

SELECT * FROM Students WHERE FirstName LIKE 'Al%';

UPDATE Students SET Address = 'Chattogram', PhoneNumber = '01955555555' WHERE StudentId = 1;

DELETE FROM Students WHERE StudentId = 1;

SELECT Gender, COUNT(*) AS TotalStudents FROM Students GROUP BY Gender;
```

Courses

```
INSERT INTO Courses (CourseName, CourseDuration, CourseHour, CreatedBy, CreatedAt)
VALUES ('Web Development', '3 months', '60 hours', 1, NOW());

SELECT * FROM Courses;

SELECT * FROM Courses WHERE CourseId = 1;

SELECT * FROM Courses WHERE CreatedBy = 1;

SELECT * FROM Courses WHERE CourseName LIKE '%Development%';

UPDATE Courses SET CourseDuration = '4 months', CourseHour = '80 hours' WHERE CourseId = 1;

DELETE FROM Courses WHERE CourseId = 1;

SELECT COUNT(*) AS TotalCourses FROM Courses;
```

Enrollment

```
INSERT INTO Enrollments (StudentId, CourseId, EnrolledBy, EnrollmentDate)
VALUES (1, 1, 1, NOW());

SELECT * FROM Enrollments;

SELECT * FROM Enrollments WHERE EnrollmentId = 1;

SELECT e.EnrollmentId, c.CourseName, e.EnrollmentDate
FROM Enrollments e
JOIN Courses c ON e.CourseId = c.CourseId
WHERE e.StudentId = 1;

SELECT e.EnrollmentId, s.FirstName, s.LastName, e.EnrollmentDate
```

```
FROM Enrollments e
JOIN Students s ON e.StudentId = s.StudentId
WHERE e.CourseId = 1;

SELECT * FROM Enrollments WHERE StudentId = 1 AND CourseId = 1;

UPDATE Enrollments SET EnrollmentDate = NOW() WHERE EnrollmentId = 1;

DELETE FROM Enrollments WHERE EnrollmentId = 1;

SELECT CourseId, COUNT(*) AS TotalEnrolled FROM Enrollments GROUP BY CourseId;
```

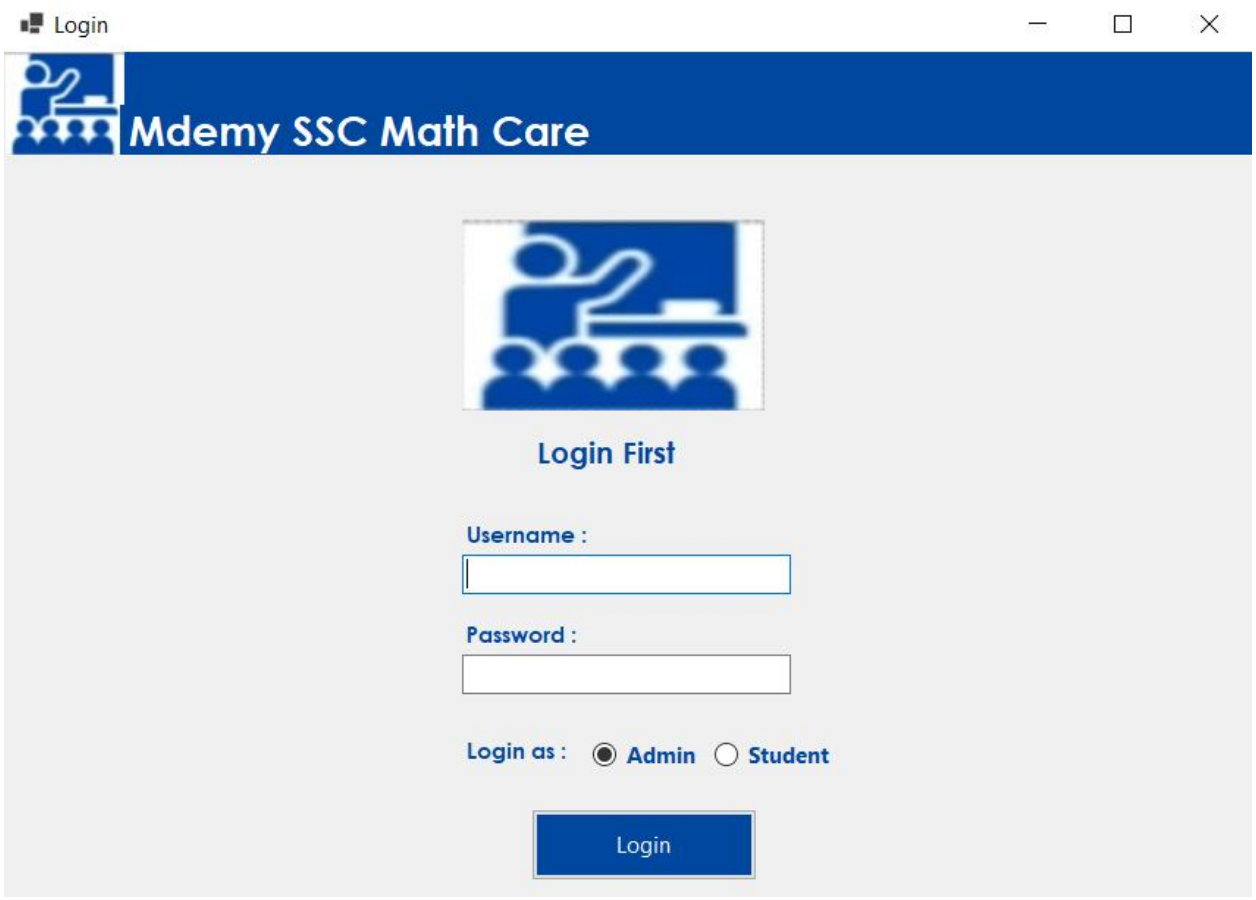
Scores

```
INSERT INTO Scores (StudentId, CourseId, Marks, AddedBy, CreatedAt)
VALUES (1, 1, 85.5, 1, NOW());

SELECT * FROM Scores;

SELECT * FROM Scores WHERE ScoreId = 1;

SELECT s.ScoreId, c.CourseName, s.Marks
FROM Scores s
JOIN Courses c ON s.CourseId = c.CourseId
WHERE s.StudentId = 1;

SELECT sc.ScoreId, st.FirstName, st.LastName, sc.Marks
FROM Scores sc
JOIN Students st ON sc.StudentId = st.StudentId
WHERE sc.CourseId = 1;

SELECT st.FirstName, st.LastName, sc.Marks
FROM Scores sc
JOIN Students st ON sc.StudentId = st.StudentId
WHERE sc.CourseId = 1
ORDER BY sc.Marks DESC
LIMIT 1;

UPDATE Scores SET Marks = 90.0 WHERE ScoreId = 1;

DELETE FROM Scores WHERE ScoreId = 1;

SELECT CourseId, AVG(Marks) AS AverageMarks FROM Scores GROUP BY CourseId;
```

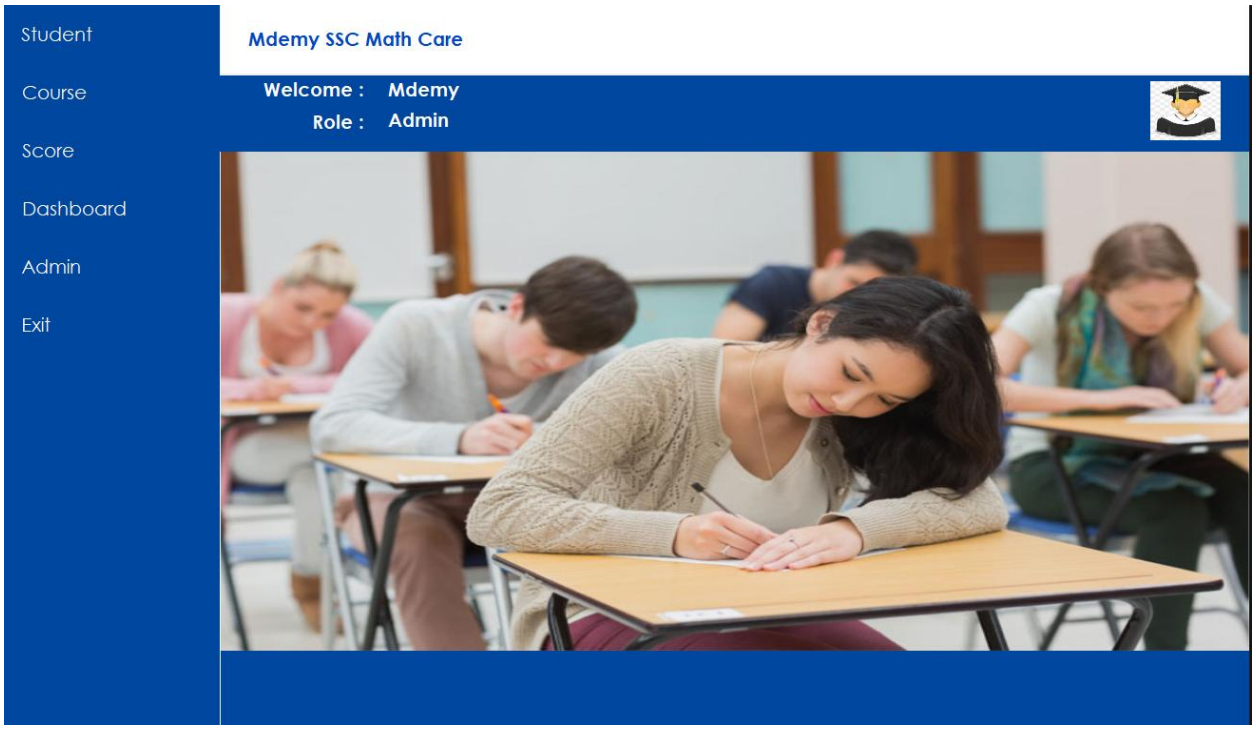
8 .UI Interface Design

1.Welcome view

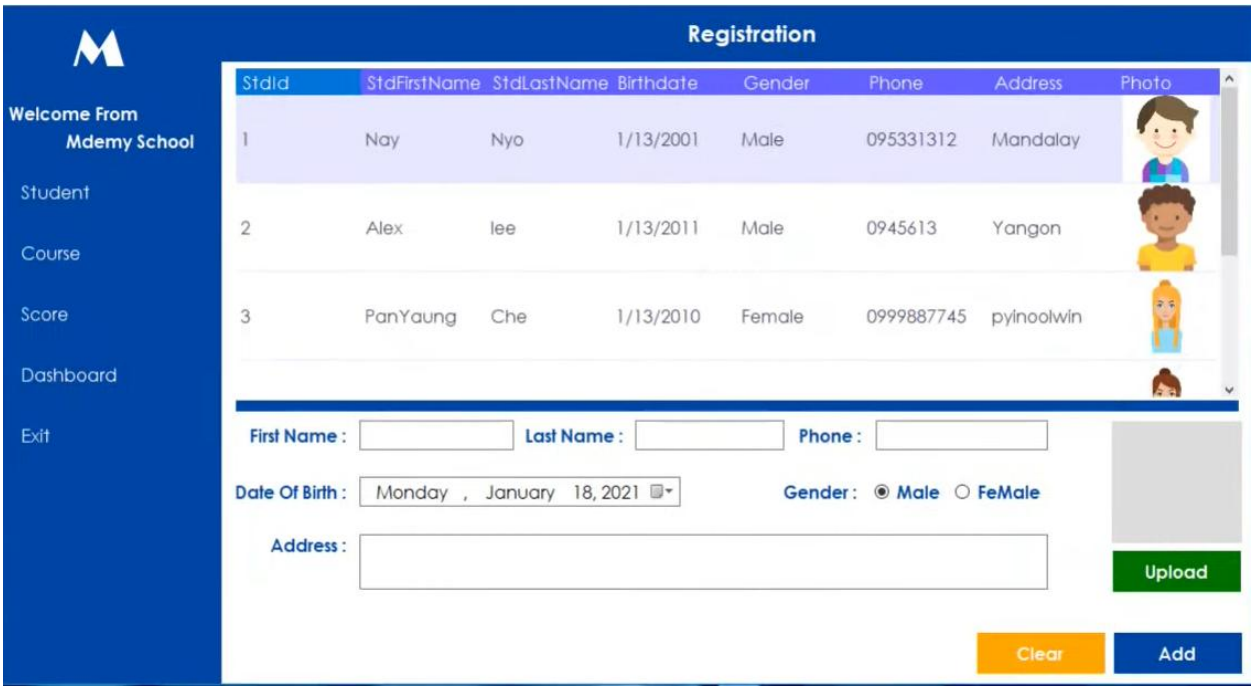
2.Login


Login
Mdemy SSC Math Care
Login First
Username :
Password :
Login as :
Admin
Student
Login

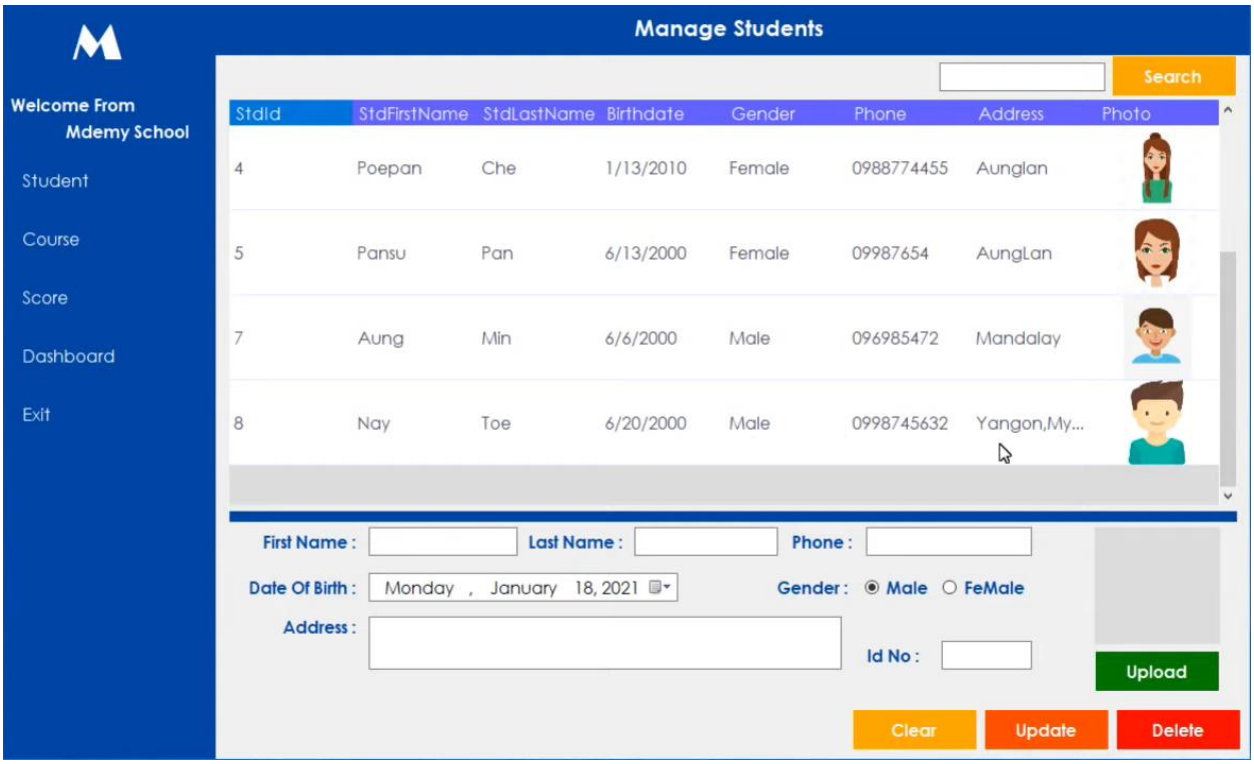
3.Dashboard



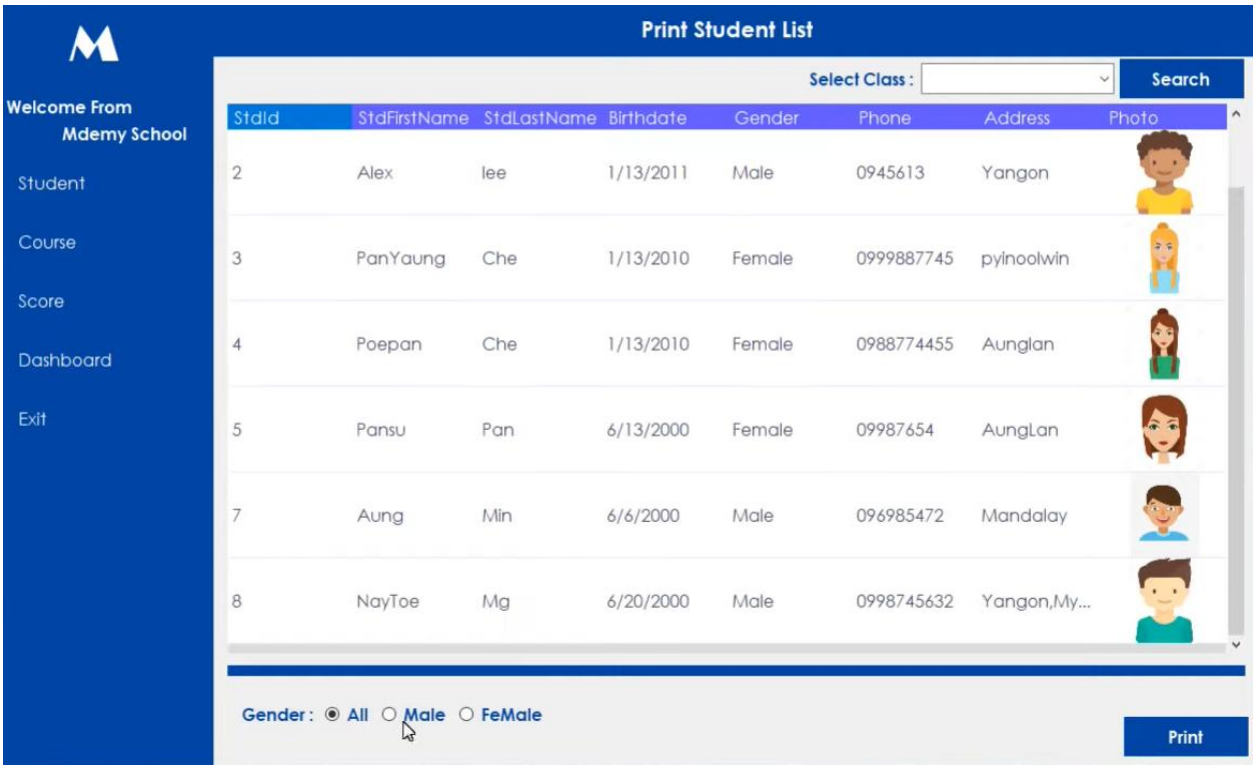
4.Registration



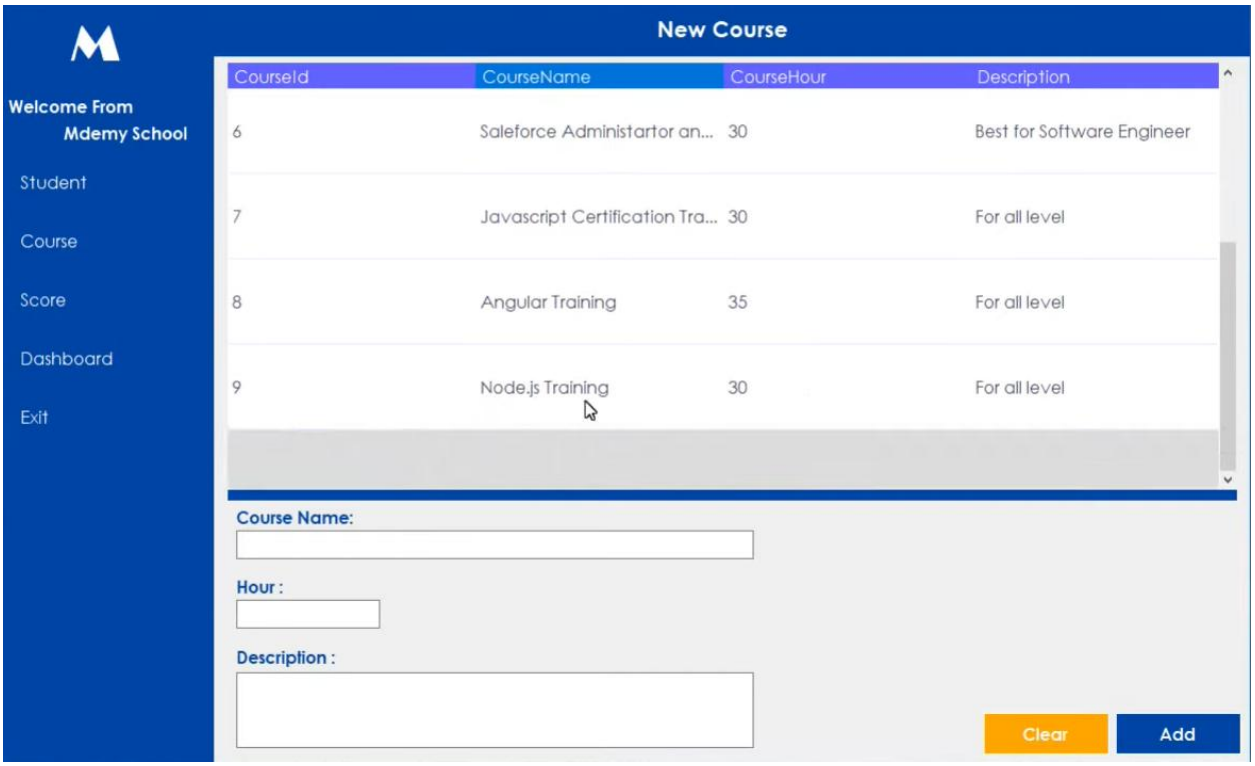
5.Manage Student



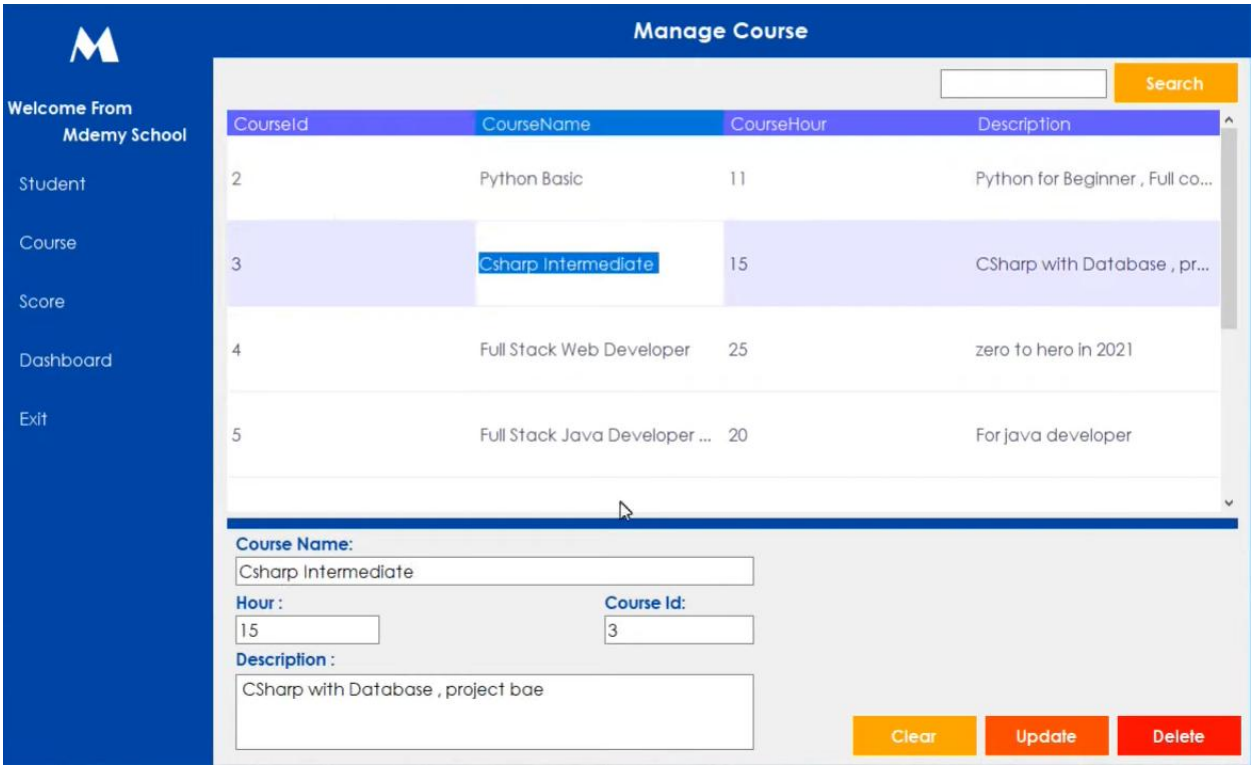
6.Print Student



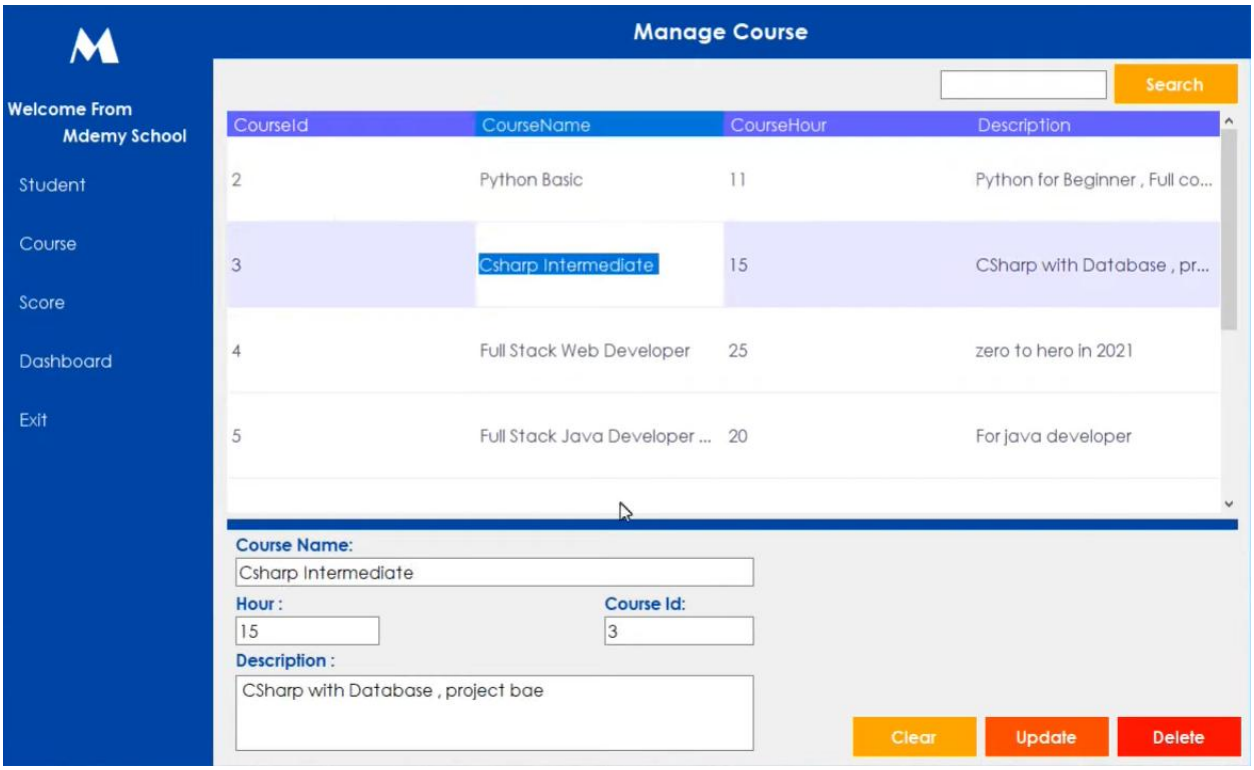
7.Add Course



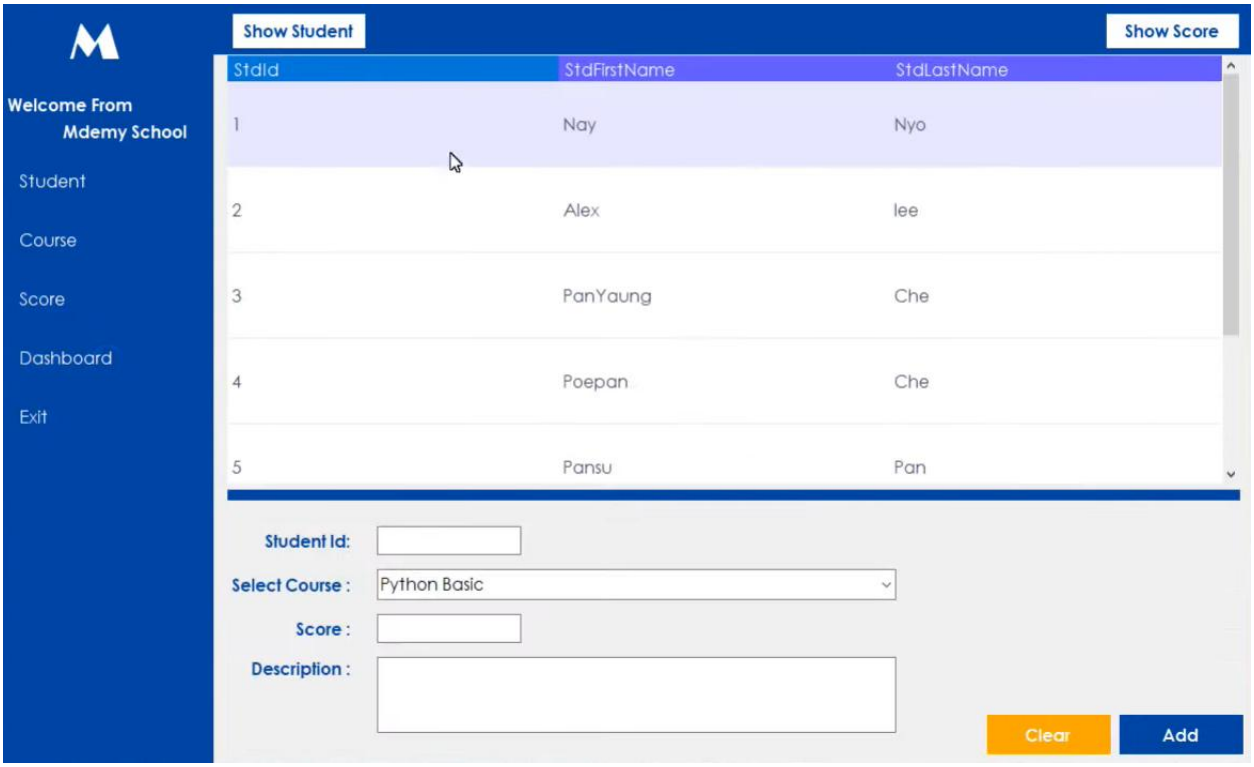
8.Mange Course



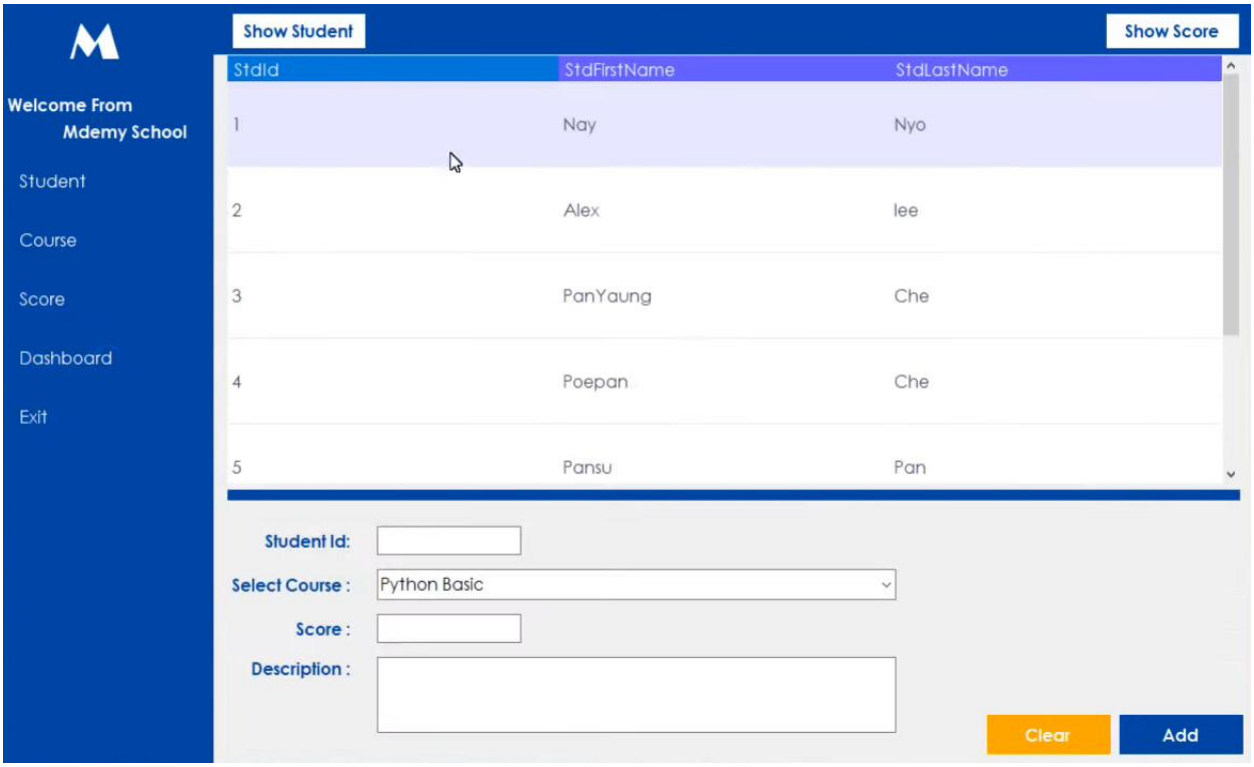
9.Print Course



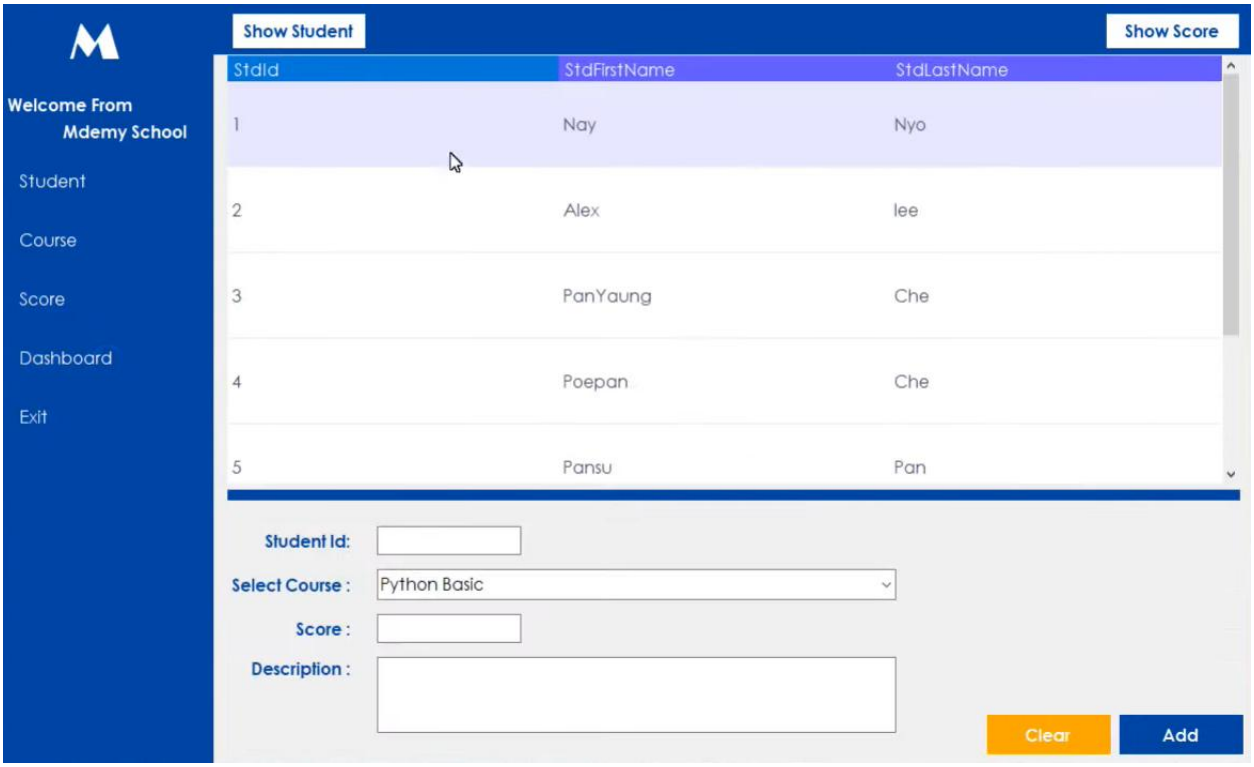
10.Add Score



11.Manage Score



12.Print Score



9.Work Distribution

Name	Id	Exact Contribution
Shahriar Nafiz Nahin	24-59162-3	Case Study & Database Design
Badhon Kabiraj	24-58369-2	Functional Requirements, User Stories, & Admin/Student Modules
Nabila Islam	24-57371-2	Course & Enrollment Management
ARIFA ASHRAFY	24-57193-2	Score Management & Final Integration

10. Conclusion & Future Work

The development of the Student Management System marks a significant milestone in modernizing academic administration by replacing error-prone manual paperwork and fragmented legacy record-keeping with a secure, centralized digital platform. Throughout this project, we successfully established a robust architecture that streamlines institutional workflows across three distinct user roles: Students, Administrators, and Super Administrators. By implementing a structured relational database adhering to Third Normal Form (3NF), the system ensures complete data integrity, eliminating redundancies while efficiently managing vital entities such as student profiles, course catalogs, academic scores, and administrative credentials.

The integration of comprehensive functional modules empowers administrators to effortlessly register learners, assign course enrollments, record precise evaluation marks, and generate essential printable reports. Simultaneously, the dedicated student portal grants learners real-time, transparent access to their personal profiles, enrolled subjects, and academic performance summaries, fostering a more engaging and accountable educational environment. Furthermore, the incorporation of hierarchical governance through the Super Admin panel ensures that system permissions and account security are maintained with the highest standards of reliability.

Ultimately, this project bridges the operational gap between institutional staff and students, drastically reducing administrative overhead and data disorganization. The successful execution of normalized database schemas, optimized SQL querying, and intuitive user interface designs proves the system's scalability and readiness for real-world deployment. As educational institutions continue to embrace digital transformation, this Student Management System stands as a solid, efficient, and extensible foundation capable of adapting to future academic demands and technological advancements.

Technology Stack & Planned Execution

Technology Stack

This Student Management System is built using a robust technology stack designed for desktop environments. The backend utilizes a relational database management system powered by **SQL** for secure, structured data storage and optimized querying. The user interface and application logic are developed using **C#** and **Windows Forms (.NET)** to provide an intuitive, responsive multi-tier architecture.

Planned Execution

The planned execution of the Student Management System project follows a structured timeline divided into four main phases: requirements gathering and system architecture design, database schema implementation and normalization, core feature development (including administrative modules, course enrollment, and scoring logic), and finally, interface integration, comprehensive testing, and project documentation finalization.